Implementation of a Modular ROS 2 Perception Pipeline for UR5 Cube Swapping

Group 29
Ali Shavakhi, Vaheid Labbaf
Email: redacted-for-privacy
Repository: `Assignment 2 Repository` Video: `Simulation Video`

***Abstract*—This report presents a modular ROS 2 solution for the autonomous swapping of two Apriltag-identified cubes using an UR5 robotic arm. The system architecture is divided into two decoupled subsystems: perception and manipulation. The perception layer robustly detects tags and estimates cube colors using an RGB camera. The manipulation layer, implemented via MoveIt 2, executes a multi-stage pick-and-place sequence utilizing a temporary buffer location. To ensure operational safety and task success, the system enforces path constraints to maintain gripper verticality and dynamically updates the planning scene with collision objects to prevent environmental contact.**

***Index Terms*—ROS2, UR5, MoveIt2, Apriltag, TF2, Perception, Camera, Color Detection**

I. INTRODUCTION

Robotic manipulation pipelines require reliable perception, consistent reference frames, and robust communication interfaces between perception and motion planning. In Assignment 2, the objective is to autonomously swap the positions of two cubes using an UR5 arm, where each cube is identified by an Apriltag placed on its top surface. An external fixed RGB camera provides the necessary visual feedback, utilizing camera intrinsics to resolve the 3D scene from 2D images.

The overall assignment requires a unified system capable of Apriltag detection, coordinate transformation, collision-free motion planning, and safe object handling. In this report, we present a comprehensive modular architecture that addresses these requirements through two main subsystems:

- A **Perception Layer** that processes raw RGB streams to extract high-fidelity state estimates. It utilizes the `dt_apriltags` library for 6-DoF pose estimation, transforming targets to the robot's `base_link` frame. Uniquely, for color classification, we implemented a geometric "band-masking" algorithm that isolates the cube's surface from the tag's black-and-white pattern, enabling robust HSV analysis even under varying lighting conditions.
- A **Manipulation Layer** that implements a robust finite state machine for the swap logic. This module utilizes a temporary buffer zone to manage the single-gripper limitation and enforces geometric path constraints to ensure the safe transport of objects without slippage. Furthermore, it integrates dynamic collision objects into the planning scene to ensure safe trajectory execution.

II. SYSTEM ARCHITECTURE

The system architecture is organized as a distributed graph of independent ROS 2 nodes, adhering to the principle of separation of concerns. This modular design ensures that the perception layer is solely responsible for state estimation, while the manipulation layer focuses on trajectory planning and execution. This decoupling allows for independent testing of vision and control subsystems.

A. Node-Level Modularity

- **TagDetectorNode:** Responsible for the 3D localization of objects. It consumes raw RGB images and camera intrinsics (f_x, f_y, c_x, c_y) to detect Apriltags using the `dt_apriltags` library. It computes the 6-DoF pose of each tag relative to the camera and utilizes a `tf2_ros::Buffer` to transform these poses into the `base_link` frame, compensating for the camera's extrinsic calibration.
- **ColorDetectorNode:** Implements a robust geometric approach for color classification. Instead of processing the full image, it uses the tag corners to generate a "band mask"—a region of interest created by subtracting the inner tag polygon from a dilated outer bounding box. This isolates the cube's colored surface from the black-and-white tag pattern, allowing for accurate hue classification in the HSV color space.
- **CubeFusionNode:** Acts as a state aggregator. It fuses pose and color streams and implements a "hold-last" policy using a time-window filter. This ensures that momentary detection dropouts (e.g., due to motion blur or arm occlusion) do not interrupt the data stream sent to the controller.
- **ManipulationNode:** The central executive node that consumes stabilized cube poses. It interfaces with the MoveIt 2 `MoveGroupInterface` for planning, dynamically manages `CollisionObject` primitives in the planning scene, and controls the gripper via an asynchronous Action Client to ensure mechanical grasp completion.

B. Communication Pattern

The system relies on standard ROS 2 interfaces to ensure robust data flow:

- **Topics:** Used for high-frequency sensor data. The manipulation node utilizes thread-safe subscriptions

(protected by ‘std::mutex‘) to synchronize the latest `/cube_a_pose` and `/cube_b_pose` messages before planning.

- **TF2 Buffer:** Used to maintain the dynamic relationship between the camera optical frame and the robot base, allowing targets to be transformed regardless of camera placement.
- **Action Interface:** The system uses `rclcpp_action` to communicate with the gripper controller, allowing the logic to block and wait until the gripper has fully opened or closed before initiating robot motion.

C. Node Summary

A structural view of the system clarifies how perception data are converted into robot actuation. Figure 1 illustrates the complete pipeline, and Table I summarizes the I/O of the architecture.

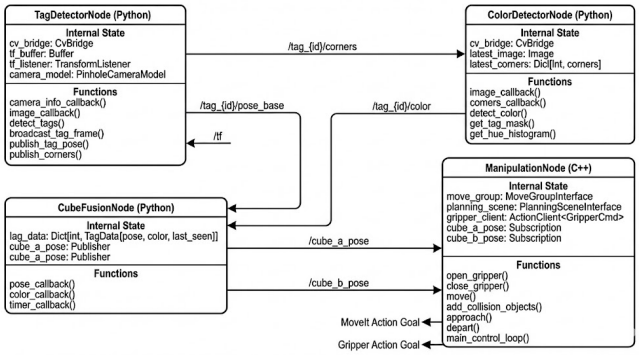


Fig. 1. Block diagram illustrating the complete pipeline from visual perception to robotic manipulation.

TABLE I
ROS 2 NODE ARCHITECTURE OVERVIEW

Node	Function	Input	Output
TagDetector	Tag pose in base	/image, TF	/tag_*/pose_base
ColorDetector	Cube color	/image, corners	/tag_*/color
CubeFusion	Stable cube poses	/tag poses	/cube_*_pose
Manipulation	Motion Control	/cube_*_pose	/joint_states, Action

III. METHODS

A. Apriltag Detection and Pose Estimation

The **TagDetectorNode** serves as the primary perception entry point. It subscribes to the RGB stream and camera intrinsics. Utilizing the `36h11` tag family, the node detects tags and solves the Perspective-n-Point (PnP) problem to estimate the 6-DoF pose of each tag (IDs 1 and 10) relative to the camera optical frame. To ensure kinematic consistency, the node computes the orientation quaternion from the rotation matrix and broadcasts the resulting transform to the TF tree.

B. TF Transformation to `base_link`

Since manipulation targets must be expressed in the robot's coordinate system, each detected tag pose is transformed from the camera optical frame to the robot's `base_link`. The node utilizes a `tf2_ros::Buffer` with a robust lookup mechanism. It first attempts to transform the pose at the exact image timestamp; if that fails due to network latency, it falls back to the latest available transform to prevent data loss.

C. Corner Extraction for ROI Definition

To facilitate efficient color processing, the detector extracts the 2D pixel coordinates of the tag's four corners. These points are packaged into a `PolygonStamped` message and published. This geometric projection defines a stable spatial reference for the color detector, ensuring the Region of Interest (ROI) remains locked onto the cube's top surface regardless of camera perspective changes.

D. Robust Color Classification

The **ColorDetectorNode** implements a geometric masking and HSV-based classification pipeline.

1) **Geometric Band-Masking:** To avoid sampling the black-and-white tag pattern, the node generates a binary mask by subtracting the inner tag polygon from a dilated outer bounding box. This isolates a "ring" of pixels representing the colored cube surface.
2) **Two-Stage HSV Analysis:** The masked ROI is converted to the HSV color space. The classifier first checks for "achromatic" colors (Black, White, Gray) based on saturation and value thresholds (e.g., $S < 35$ or $V < 70$). If the pixel is chromatic, it contributes to a Hue histogram.
3) **Voting Mechanism:** The final color label is determined by the peak of the Hue histogram, filtering out noise and ensuring robust classification even under varying lighting conditions.

E. Fusion and Dropout Handling

Real-world detection is prone to intermittent dropouts due to motion blur or arm occlusion. The **CubeFusionNode** aggregates raw poses and applies a time-domain filter. It implements a "hold-last" policy with a configurable timeout (e.g., 0.6s validity, 3.0s backup hold), ensuring that the manipulation node receives a continuous stream of valid target poses even during brief perception gaps.

F. Manipulation Strategy

The manipulation logic is encapsulated in the `ManipulationNode`, which orchestrates the pick-and-place sequence using MoveIt 2 and a finite state machine logic.

1) Dynamic Planning Scene: To ensure safety, the node actively modifies the MoveIt planning scene. It spawns collision objects representing the physical environment:

- `box1` and `box2`: Dynamic collision boxes generated at the detected coordinates of Cube A and Cube B to prevent the robot from colliding with the objects it is not currently grasping.
- `box3`: A static collision box representing the "Buffer Zone" (safe intermediate location).

These objects are added/removed dynamically during the execution sequence to allow grasping while preventing collision during transit.

2) Action-Based Gripper Control: We utilize an Action Client (`rclcpp_action::Client<GripperCommand>`) to interface with the gripper controller. Unlike fire-and-forget topic publication, the Action interface allows the node to block execution until the gripper has achieved the desired width or effort, preventing timing hazards (e.g., moving the arm before the grasp is secure).

3) Constrained Path Planning: A critical requirement is to prevent the cubes from slipping due to centrifugal forces or tilting. We enforce `moveit_msgs::OrientationConstraint` on the end-effector link (`tool0`). The constraints restrict roll and pitch deviations to within 0.9 radians while allowing free yaw rotation. This forces the OMPL planner to generate "upright" trajectories, ensuring a stable transport phase.

4) Buffered Swap Logic: Since the UR5 is equipped with a single gripper, a direct swap is impossible. The node implements a three-stage buffered logic:

1) **A $\rightarrow$ Buffer:** Pick Cube A and place it at the safe intermediate waypoint (`box3`).
2) **B $\rightarrow$ A:** Pick Cube B and place it at Cube A's original location.
3) **Buffer $\rightarrow$ B:** Pick Cube A from the buffer and place it at Cube B's original location.

Motion primitives (`approach`, `move`, `depart`) are used for each segment to ensure vertical lifting and lowering.

IV. EXPERIMENTAL RESULTS

A. Perception Performance

The perception pipeline demonstrated high robustness in repeated simulation trials.

- **Tag Localization:** The `TagDetectorNode` consistently solved the PnP problem, providing stable 6-DoF poses transformed to the `base_link` frame via the TF tree.
- **Color Classification:** The geometric "band-masking" strategy proved highly effective. By explicitly excluding the black-and-white tag center, the HSV classifier avoided false "gray/black" detections and correctly identified the cube's hue even under varying lighting angles.
- **Tracking Stability:** The `CubeFusionNode`'s "hold-last" policy successfully bridged detection gaps. During the "approach" and "grasp" phases, where the robot arm partially occluded the camera's view of the tag, the fusion node maintained the target publication (using the configured backup window), preventing the planner from aborting due to target loss.

B. Manipulation Execution

The manipulation subsystem successfully executed the three-stage swap sequence (Cube A $\rightarrow$ Buffer, Cube B $\rightarrow$ Target A, Buffer $\rightarrow$ Target B) without mechanical failure.

- **Path Constraints & Safety:** The application of `moveit_msgs::OrientationConstraint` was critical for success. In initial unconstrained tests, the OMPL planner occasionally generated "scooping" trajectories with excessive roll/pitch, causing the simulated cube to slip from the gripper. With constraints enabled (tolerance < 0.9 rad), the end-effector maintained a strict vertical orientation during all transport phases.
- **Dynamic Collision Avoidance:** The dynamic management of collision objects (`box1`, `box2`, and the buffer zone `box3`) effectively updated the planning scene in real-time. This ensured that the arm planned trajectories around the stationary cube while transporting the active one, preventing inter-object collisions.

V. CONCLUSION

In this work, we presented a comprehensive and modular ROS 2 pipeline for the autonomous manipulation of Apriltag-identified objects. The proposed architecture successfully decoupled state estimation from control logic, allowing for robust performance even under perception uncertainty.

On the perception side, the novel geometric "band-masking" technique effectively neutralized tag-pattern noise, ensuring accurate color classification, while the temporal fusion layer bridged detection gaps caused by occlusions. On the manipulation side, the system overcame the kinematic and physical limitations of the single-gripper UR5 configuration through a buffered swap logic. Furthermore, the rigorous application of `MoveIt 2` path constraints and dynamic collision objects ensured operational safety, preventing object slippage and environmental collisions.

Ultimately, this project demonstrates the critical importance of integrating constraint-aware motion planning with robust vision pipelines to achieve reliable autonomous manipulation in unstructured environments.

ACKNOWLEDGEMENTS

AI usage disclosure: Portions of this report text were supported by AI-based language models for improving clarity and formatting. All system architecture decisions, perception logic, and ROS 2 code were designed and implemented by the authors.